Executing SQL Stored Procedures from inside a Web Application Part 1

By Don Schlichting

This article will explore the hows and whys of executing SQL Stored Procedures from inside a web application.

Introduction

There are so many good reasons to use SQL Stored Procedures from inside web applications (as opposed to passing a query string), that the benefits of working with them will far outweigh any learning curve required. Moreover, the leaning curve is small (Really, it is). Some of the benefits of stored procedures include:

- Increased Sever Performance. Because stored procedures are precompiled inside SQL Server, they usually execute quicker than passing in query strings.
- Increased Security. Users can be given permission to only execute the stored procedures, not directly access the underlying data.
- Injection Attack Prevention. An injection attack occurs when malicious code is passed in as a string and parsed by the SQL Server engine. Stored procedures prevent this from happening. See the Microsoft article "Stop SQL Injection" at http://msdn.microsoft.com/msdnmag/issues/04/09/SQLInjection/default.aspx for a complete description and explanation of injection attacks.
- Portability. If your application may ever need to run on multiple databases, it is easier to convert a group of centrally located stored procedures than it is to hash through all the code of an application looking for SQL statements.
- Performance on Datagrids and Tree Views. Many datagrid and tree view tools do not offer paging (The ability to only see the first handful of records, then additional handfuls on a button click). Or, if they do page, it is implemented by bringing down all the records from the server to the client, then only displaying part of them. If the result set is large, this can be a huge performance issue. By using stored procedures, it is easy to only bring down the rows truly needed by the grid or tree, then fetch additional records as requested.
- Multiple Return Types. Many times, in addition to needing a datagrid or result set of rows, the aggregate or calculated total of a column is also needed. An example would be a Windows Form showing sales, where both the line items and a grand total is shown. Without stored procedures, the options are usually to make two database calls, one for rows and one for the total, or sequentially read the rows into the form and make a calculation as it is displayed. Stored procedures allow discreet values, such as Total Sales in this example, to be returned as a separate piece of information along with the detail rows in only one database call.
- MDAC ODBC problems. If transactional control or locking is required for a statement, there may be limitations when invoked from inside query strings. Stored procedures avoid this.
- Organization and Code Reuse. Stored Procedures are centrally stored inside SQL Server, making it easy to find, share, and reuse code. In addition, SQL Servers increased support for Schemas help with additional naming and organization convention options.
- **Learning Curve**
- So how much leaning curve is there to use Stored Procedures? Surprisingly, very little. If you can write a query string, you can write a stored procedure (It is the

same code). Stored Procedures use the same connection string security options as query string statements do, so there is no changes with the connection object. We will add only a couple of new stored procedure lines of code to the command object, and that's it.

- **Examples**
- These examples use C# to execute stored procedures located inside the SQL Server database Adventure Works. The same code will also work on SQL Sever. Visual Studio was used to develop the web site. If you do not have Visual Studio, a Free light edition called Web Matrix is available from Microsoft at http://www.asp.net/webmatrix/Default.aspx?tabindex=0&tabid=1 .
- **Creating the Stored Procedure**
- In this example, sales from the Adventure Works sample database will be returned. A datagrid will be populated with each line item for a specific product ID. The first step is to create the stored procedure in SQL Server. Open a new query window and enter the following statement:
- USE AdventureWorks;
- GO
- CREATE PROCEDURE testProc1
- AS
- SELECT OrderQty, LineTotal
- FROM Sales.SalesOrderDetail
- WHERE ProductID = '744';
- The basic syntax is very straightforward. The name of the new procedure follows the "CREATE PROCEDURE" key words. After the "AS", the actual statement is entered. The procedure can also be created graphically from inside SQL Server Management Studio by expanding Databases, AdventureWorks, Programmability, then right clicking on Stored Procedures, and finally select New Stored Procedure. This method brings up an intimidating template to be filled in.
- **Creating the Web Page**
- From inside Visual Studio, create a new aspx web page. Drag and drop a grid view onto the web page. After, the source section of the page should resemble the following.
- <html xmlns="http://www.w3.org/1999/xhtml" >
- <head runat="server">
- <title>Test 1</title>
- </head>
- <body>
- <form id="form1" runat="server">
-
-
- <asp:GridView ID="GridView1" runat="server">
- </asp:GridView>
-
-
- </form>
- </body>
- </html>
- **SQL Client Name Space**
- Next, above the previous section, create a script area to execute the new stored procedure at page load. The key words needed to execute the stored procedure are located in the System Data or System Data SqlClient name spaces.
- <%@ Page Language="C#" %>
- <script runat="server">

- protected void Page_Load(object sender, EventArgs e)
- {
- System.Data.SqlClient.SqlConnection objConn = new System.Data.SqlClient.SqlConnection();
- objConn.ConnectionString = "server=.; database=AdventureWorks; uid=sa;pwd=test;";
- objConn.Open();
- System.Data.SqlClient.SqlCommand objCmd = new System.Data.SqlClient.SqlCommand("testProc1", objConn);
- objCmd.CommandType = System.Data.CommandType.StoredProcedure;
- GridView1.DataSource = objCmd.ExecuteReader();
- GridView1.DataBind();
- objConn.Close();
- }
- </script>
- Save and view the page. A small result set should be returned.

OrderQty	LineTotal
1	809.760000
2	1619.520000
1	809.760000
1	809.760000
1	809.760000
1	809.760000
2	1619.520000
1	809.760000
2	1619.520000
1	809.760000
1	809.760000
1	809.760000
2	1619.520000

The syntax for executing the stored procedure is very similar to executing a query string.

The first line creates a new SQL connection:

System.Data.SqlClient.SqlConnection objConn = new
System.Data.SqlClient.SqlConnection();

In the next line, server specific information is passed to the connection. In this example, sa and its password were used. Windows Integrated security could also have been used, but we would need to add some additional permissions to our stored procedure first. There will be an example of this in part two. For this example, the SQL Server is local, on the same machine as the web server, so a single period is used as the server name.

```
objConn.ConnectionString = "server=.; database=AdventureWorks; uid=sa; pwd=test;";
```

The connection is opened:

```
    objConn.Open();
```

These next two lines direct a command object to use our new stored procedure. The name of the procedure and the conneciton object name are passed in, then the command object is told that the name just provided is a stored procedure.

```
System.Data.SqlClient.SqlCommand objCmd = new
System.Data.SqlClient.SqlCommand("testProc1", objConn);
objCmd.CommandType = System.Data.CommandType.StoredProcedure;
```

Now the grid is bound to the command object and executed. Afterwords, the conneciton is closed.

```
    GridView1.DataSource = objCmd.ExecuteReader();
    GridView1.DataBind();
    objConn.Close();
```

Conclusion

Executing Stored Procedures from web pages provides numerous benefits and are easy to code. In the next article, we will use Windows Integrated Security and pass parameters (variables) in and out of stored procedures.